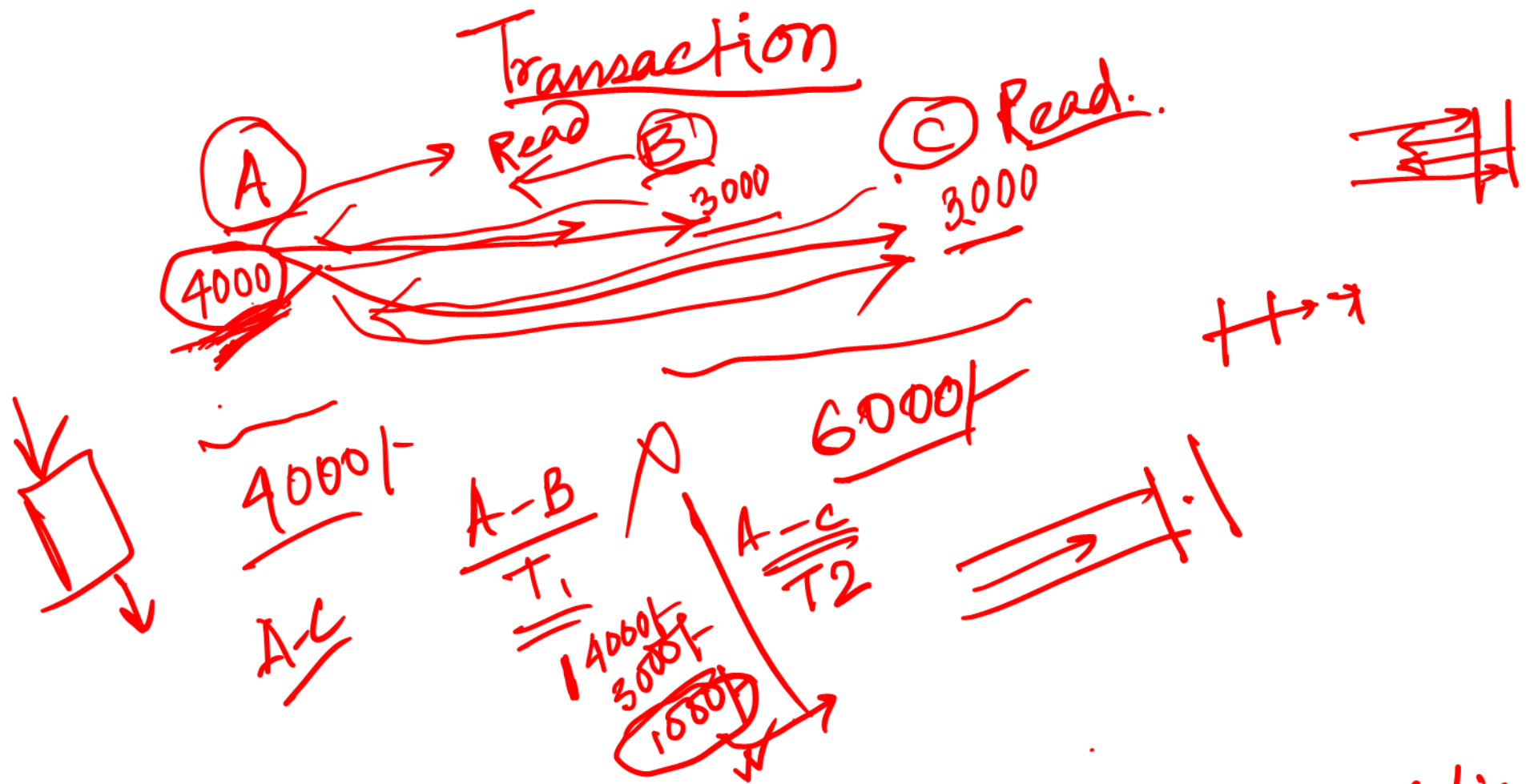




Transaction is an unit of program written in some high level language like C, C++, Java, etc and possibly some update commands are present.

Properties of transaction: ACID Property

A → Atomicity

C → Consistency

I → Isolation

D → Durability

→ Either all, or none anything.

Two transaction can't be execute concurrently due to consistency of the database until they are isolated.

→ If data transaction committed / completed successfully →

A-B ✓

C-D ✓

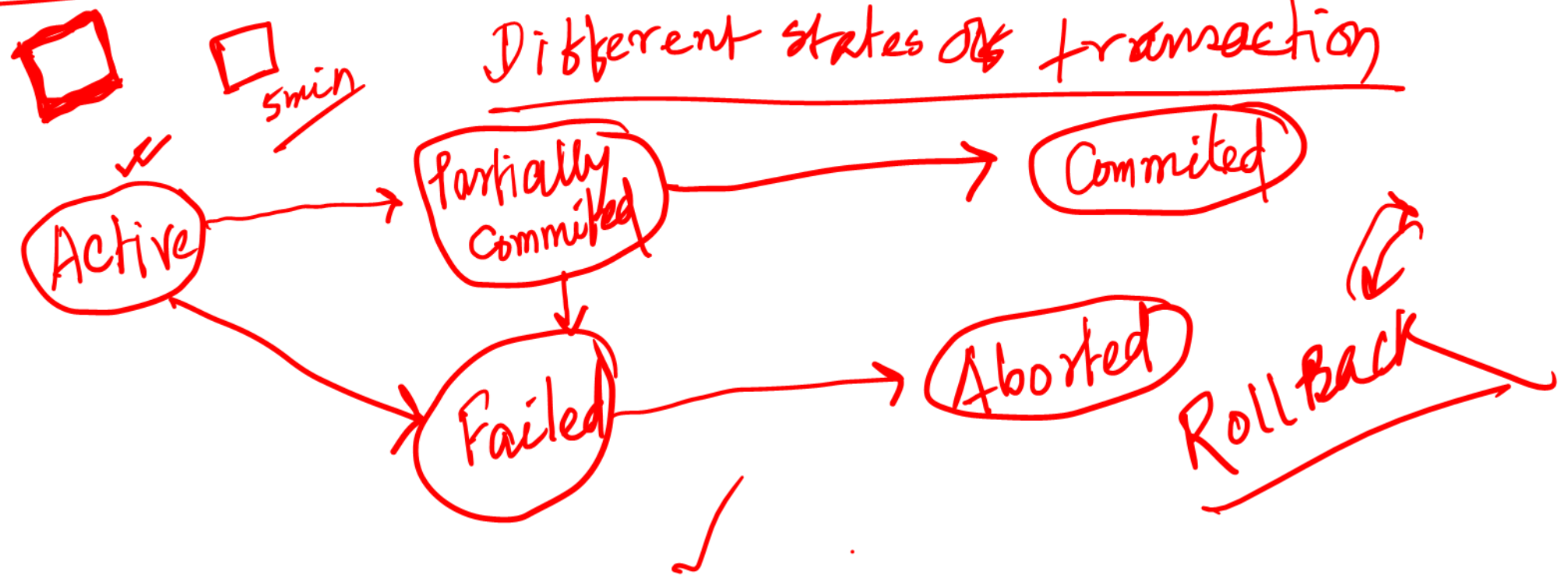
5 ✓

50

Transaction operation

$\left\{ \begin{array}{l} \rightarrow \text{Read} \\ \rightarrow \text{Write} \end{array} \right\}$

$\frac{A-B}{\text{Read } A}$
 $\frac{A=A-B}{\text{Write } A}$



Serializability

If transactions are executed serially that means after completion of one transaction the next transaction is started, then it is called serializability.

- (i) Conflict Serializability
- (ii) View Serializability

Conflict Serializability: $\rightarrow$ Let T_i & T_j be two consecutive instructions belonging to different transactions T_i & T_j respectively. They are said to be conflict if they operate on same data item and at least one write operation is present.

That means if T_i & T_j operates on different data items then they are not conflict. If T_i & T_j operate on same data item, one of the following

4-cases may arises —

Case-1: $\rightarrow$ $\begin{matrix} \checkmark T_i \Rightarrow \text{Read}(A) \\ \checkmark T_j \Rightarrow \text{Read}(A) \end{matrix} \left. \vphantom{\begin{matrix} \checkmark T_i \Rightarrow \text{Read}(A) \\ \checkmark T_j \Rightarrow \text{Read}(A) \end{matrix}} \right\}$ since both are read operation, so, They are not conflict.

Case-2: →

$T_i \Rightarrow \text{Read}(Q) -$

$T_j \Rightarrow \text{Write}(Q) -$

$\{Q = 5555\}$
 $Q = Q - 1000$
 $\text{Read}(Q)$
 $\text{Write}(Q)$

If T_i execute first it can read the old value of Q . T_j execute after T_i it can read the update value of Q . So, order matters. Therefore they are conflict.

Case-3: →

$T_i \Rightarrow \text{Write}(Q) \sim$

$T_j \Rightarrow \text{Read}(Q) \sim$

S

Same as above

Case-9! $\rightarrow T_i \Rightarrow \text{write}(Q) \Rightarrow$
 $T_j \Rightarrow \text{write}(Q).$

Initially they are not-conflict, 'but next read operation Reads the last write operation value, therefore they are conflict.

T_1

$R_1(A)$
 $W_1(A)$

T_2
 $R_2(C)$
 $R_2(B)$
 $W_2(B)$

T_3

$R_3(B)$
 $R_3(C)$

$W_3(B)$
 $W_3(C)$

$R_2(A)$

$R_1(B)$
 $W_1(B)$

$W_2(A)$

Ex-2! →

T_1
 $R(A)$
 $W(A)$
 $R(B)$
 $W(B)$

T_2

$R(C)$
 $W(C)$
 $R(B)$
 $W(B)$

→ Not Conflict.

Ex-3 :->

T₁
R(A)

R(C)

W(A)

T₂
R(A)

W(C)

W(A)

<u>T₁</u>	<u>T₂</u>
R(A) R(C) <u>W(A)</u>	<u>R(A)</u> W(C) W(A)

Conflict

View Serializability

Next Date